

2026

STD621-FA 1

WILLEM GERBER

In line with industry requirements, CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/TE07/004) and provisionally registered as a private higher education institution (only selected registered campuses) (No 2014/HED7/004).

Directors: J Newby, E Ferreira, B Brown. | Co. Reg No: 2005/038944/07

ctutrainig.ac.za | 0861 100 395 | enquiry@ctutrainig.co.za

SECTION A: Short Answer Questions

Question 1

Program.cs:

```

Program.cs

using System;
using System.Globalization;

namespace StudentResults
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter student name: ");
            string studentName = Console.ReadLine() ?? string.Empty;

            double[] marks = new double[3];

            for (int i = 0; i < 3; i++)
            {
                while (true)
                {
                    Console.WriteLine($"Enter mark for Subject {i + 1}: ");
                    string input = Console.ReadLine() ?? string.Empty;

                    if (double.TryParse(input, NumberStyles.Number, CultureInfo.CurrentCulture, out marks[i]))
                        break;

                    Console.WriteLine("Invalid input. Please enter a numeric value.");
                }
            }

            double total = marks[0] + marks[1] + marks[2];
            double average = total / 3.0;
            string result = average >= 50 ? "PASS" : "FAIL";
            string issuedAt = DateTime.Now.ToString("dd MMM yyyy HH:mm:ss");

            Console.WriteLine();
            Console.WriteLine("===== STUDENT RESULTS =====");
            Console.WriteLine($"Student Name: {studentName}");
            Console.WriteLine($"Total Marks: {total}");
            Console.WriteLine($"Average Marks: {average:N1}");
            Console.WriteLine($"Result: {result}");
            Console.WriteLine($"Result Issued At: {issuedAt}");
            Console.WriteLine();
            Console.WriteLine("Press any key to exit... ");
            Console.ReadKey();
        }
    }
}

```

Output Screenshots:

```

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\wgerb\Desktop\SDT-621-FA1>dotnet run --project "c:\Users\wgerb\Desktop\SDT-621-FA1\Question1"
Enter student name: Willem
Enter mark for Subject 1: 79
Enter mark for Subject 2: 60
Enter mark for Subject 3: 82

===== STUDENT RESULTS =====
Student Name: Willem
Total Marks: 221
Average Marks: 73,7
Result: PASS
Result Issued At: 28 Apr 2026 16:55:44

Press any key to exit...

```

Question 2

```

Program.cs

using System;
using System.Globalization;

namespace SimpleATM
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("===== CTU SIMPLE ATM SYSTEM =====");
            Console.WriteLine();
            Console.WriteLine("HI , WHAT IS YOUR NAME?");
            string name = Console.ReadLine() ?? string.Empty;

            Console.WriteLine();
            Console.WriteLine($"WELCOME {name.ToUpper()}!");

            decimal balance;
            while (true)
            {
                Console.Write("Enter account balance: ");
                if (decimal.TryParse(Console.ReadLine(),
                    NumberStyles.Number, CultureInfo.CurrentCulture, out balance))
                    break;
                Console.WriteLine("Invalid input. Please enter a numeric
value.");
            }

            decimal withdrawal;
            while (true)
            {
                Console.Write("Enter withdrawal amount: ");
                if (decimal.TryParse(Console.ReadLine(),
                    NumberStyles.Number, CultureInfo.CurrentCulture, out withdrawal))
                    break;
                Console.WriteLine("Invalid input. Please enter a numeric
value.");
            }

            Console.WriteLine();

            if (withdrawal <= 0)
            {
                Console.WriteLine("Withdrawal amount must be greater than
zero.");
            }
            else if (withdrawal > balance)
            {
                Console.WriteLine("Insufficient funds. Transaction
declined.");
            }
            else
            {
                decimal updatedBalance = balance - withdrawal;
                string transactionTime = DateTime.Now.ToString("dd MMM yyyy
HH:mm:ss");

                Console.WriteLine("Withdrawal successful!");
                Console.WriteLine($"Updated Balance: {updatedBalance:N2}");
                Console.WriteLine($"Transaction Time: {transactionTime}");
            }
        }
    }
}

```

Output Screenshots:

```

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\wgerb\Desktop\SDT-621-FA1>dotnet run --project "c:\Users\wgerb\Desktop\SDT-621-FA1\Question2"
===== CTU SIMPLE ATM SYSTEM =====

HI , WHAT IS YOUR NAME?
Willem

WELCOME WILLEM!
Enter account balance: 5000
Enter withdrawal amount: 250

Withdrawal successful!
Updated Balance: 4 750,00
Transaction Time: 28 Apr 2026 17:01:45

C:\Users\wgerb\Desktop\SDT-621-FA1>

```

Question 3:

Program.cs:

```

Program.cs

namespace Question3;

public partial class Form1 : Form
{
    private const string Placeholder = "Enter programming language";

    public Form1()
    {
        InitializeComponent();
    }

    private void txtLanguage_Enter(object sender, EventArgs e)
    {
        if (txtLanguage.Text == Placeholder)
        {
            txtLanguage.Text = "";
            txtLanguage.ForeColor = Color.Black;
        }
    }

    private void txtLanguage_Leave(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(txtLanguage.Text))
        {
            txtLanguage.Text = Placeholder;
            txtLanguage.ForeColor = Color.Gray;
        }
    }

    private void btnAdd_Click(object sender, EventArgs e)
    {
        string language = txtLanguage.Text.Trim();

        if (string.IsNullOrEmpty(language) || language == Placeholder)
        {
            MessageBox.Show("Please enter a programming language.", "Empty
Input", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

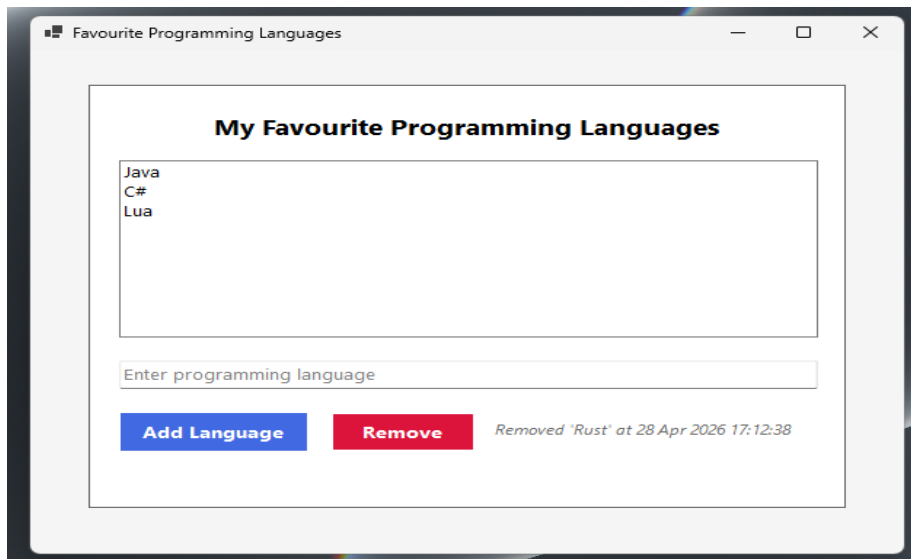
        foreach (var item in lstLanguages.Items)
        {
            if (item.ToString()!.Equals(language,
StringComparison.OrdinalIgnoreCase))
            {
                MessageBox.Show($"'{language}' is already in the list.",
"Duplicate", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }
        }

        lstLanguages.Items.Add(language);
        txtLanguage.Text = Placeholder;
        txtLanguage.ForeColor = Color.Gray;
        lblStatus.Text = "";
    }

    private void btnRemove_Click(object sender, EventArgs e)
    {
        if (lstLanguages.SelectedItem == null)
        {
            MessageBox.Show("Please select a language to remove.", "No
Selection", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        string removed = lstLanguages.SelectedItem.ToString();
        lstLanguages.Items.Remove(lstLanguages.SelectedItem);
        lblStatus.Text = $"Removed '{removed}' at {DateTime.Now:dd MMM yyyy
HH:mm:ss}";
    }
}

```

Output Screenshots:**SECTION B: Long Answer Questions****Question 1: Emfuleni Municipality Console Application****Resident.cs:**

```
Program.cs

namespace Question5
{
    public class Resident
    {
        public string Name { get; set; }
        public string Address { get; set; }
        public string AccountNumber { get; set; }
        public double MonthlyUtilityUsage { get; set; }

        public Resident(string name, string address, string accountNumber, double monthlyUtilityUsage)
        {
            Name = name;
            Address = address;
            AccountNumber = accountNumber;
            MonthlyUtilityUsage = monthlyUtilityUsage;
        }
    }
}
```

Program.cs:

Program.cs

```
using Question5;

Console.WriteLine("=== Welcome to Emfuleni Municipality Service Desk ===");

Console.Write("How many residents do you want to register? ");
int residentCount = int.Parse(Console.ReadLine());

List<Resident> residents = new List<Resident>();

for (int i = 1; i ≤ residentCount; i++)
{
    Console.WriteLine($"\\n--- Resident {i} ---");
    Console.Write("Name: ");
    string name = Console.ReadLine();
    Console.Write("Address: ");
    string address = Console.ReadLine();
    Console.Write("Account Number: ");
    string accountNumber = Console.ReadLine();
    Console.Write("Monthly Utility Usage (kWh or litres): ");
    double utilityUsage = double.Parse(Console.ReadLine());

    residents.Add(new Resident(name, address, accountNumber, utilityUsage));
}

Console.Write("\\nHow many service requests do you want to log? ");
int requestCount = int.Parse(Console.ReadLine());

List<ServiceRequest> serviceRequests = new List<ServiceRequest>();

for (int i = 1; i ≤ requestCount; i++)
{
    Console.WriteLine($"\\n--- Service Request {i} ---");
    Console.Write($"Select resident by number (1 to {residentCount}): ");
    int residentIndex = int.Parse(Console.ReadLine()) - 1;

    Console.Write("Request Type (e.g., Water Outage, Burst Pipe): ");
    string requestType = Console.ReadLine();

    Console.Write("Priority Level (1-5): ");
    int priority = int.Parse(Console.ReadLine());

    Console.Write("Severity Level (1-10): ");
    int severity = int.Parse(Console.ReadLine());

    Console.Write("Estimated Resolution Hours: ");
    int hours = int.Parse(Console.ReadLine());

    serviceRequests.Add(new ServiceRequest(residents[residentIndex], requestType, priority, severity,
hours));
}

UtilitiesManager manager = new UtilitiesManager();

foreach (var request in serviceRequests)
{
    manager.GenerateServiceReport(request);
}

manager.GenerateFinalSummary(serviceRequests);

Console.WriteLine("\\nThank you for using the Emfuleni Municipality Service Desk.");
```

UtilitiesManager.cs:

```

namespace Question5
{
    public class UtilitiesManager
    {
        public int CalculateUrgencyScore(ServiceRequest request)
        {
            return request.PriorityLevel * request.SeverityLevel * 2;
        }

        public void GenerateServiceReport(ServiceRequest request)
        {
            int urgencyScore = CalculateUrgencyScore(request);
            int adjustedResolution = request.EstimatedResolutionHours + request.PriorityLevel;
            double householdImpactScore = request.Resident.MonthlyUtilityUsage * request.SeverityLevel /
10.0;

            Console.WriteLine("\n==== Service Report =====");
            Console.WriteLine($"Resident: {request.Resident.Name}");
            Console.WriteLine($"Service Type: {request.RequestType}");
            Console.WriteLine($"Urgency Score: {urgencyScore}");
            Console.WriteLine($"Adjusted Resolution: {adjustedResolution} hours");
            Console.WriteLine($"Household Impact Score: {householdImpactScore:F2}");
        }

        public void GenerateFinalSummary(List<ServiceRequest> requests)
        {
            ServiceRequest highest = requests.OrderByDescending(r => CalculateUrgencyScore(r)).First();
            int urgencyScore = CalculateUrgencyScore(highest);
            int adjustedResolution = highest.EstimatedResolutionHours + (highest.SeverityLevel /
highest.PriorityLevel);
            double householdImpactScore = highest.Resident.MonthlyUtilityUsage * highest.SeverityLevel /
10.0;

            Console.WriteLine("\n==== FINAL MUNICIPAL SUMMARY =====");
            Console.WriteLine("Highest priority issue:");
            Console.WriteLine($"Resident: {highest.Resident.Name}");
            Console.WriteLine($"Service Type: {highest.RequestType}");
            Console.WriteLine($"Urgency Score: {urgencyScore}");
            Console.WriteLine($"Adjusted Resolution: {adjustedResolution} hours");
            Console.WriteLine($"Household Impact Score: {householdImpactScore:F2}");
        }
    }
}

```

ServiceRequest.cs:

```

namespace Question5
{
    public class ServiceRequest
    {
        public Resident Resident { get; set; }
        public string RequestType { get; set; }
        public int PriorityLevel { get; set; }
        public int SeverityLevel { get; set; }
        public int EstimatedResolutionHours { get; set; }

        public ServiceRequest(Resident resident, string requestType, int priorityLevel, int
severityLevel, int estimatedResolutionHours)
        {
            Resident = resident;
            RequestType = requestType;
            PriorityLevel = priorityLevel;
            SeverityLevel = severityLevel;
            EstimatedResolutionHours = estimatedResolutionHours;
        }
    }
}

```

Output Screenshots:

```

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\wgerb\Desktop\SDT-621-FA1>dotnet run --project "c:\Users\wgerb\Desktop\SDT-621-FA1\Question5"
=== Welcome to Emfuleni Municipality Service Desk ===
How many residents do you want to register? 2

--- Resident 1 ---
Name: Willem
Address: 41 Goud Street
Account Number: 00123
Monthly Utility Usage (kWh or litres): 45

--- Resident 2 ---
Name: Aiden
Address: 43 Goud Street
Account Number: 00124
Monthly Utility Usage (kWh or litres): 29

How many service requests do you want to log? 1

--- Service Request 1 ---
Select resident by number (1 to 2): 1
Request Type (e.g., Water Outage, Burst Pipe): Power Outage
Priority Level (1-5): 5
Severity Level (1-10): 8
Estimated Resolution Hours: 2

==== Service Report ====
Resident: Willem
Service Type: Power Outage
Urgency Score: 80
Adjusted Resolution: 7 hours
Household Impact Score: 36,00

==== FINAL MUNICIPAL SUMMARY ====
Highest priority issue:
Resident: Willem
Service Type: Power Outage
Urgency Score: 80
Adjusted Resolution: 3 hours
Household Impact Score: 36,00

Thank you for using the Emfuleni Municipality Service Desk.

C:\Users\wgerb\Desktop\SDT-621-FA1>

```

Question 2: Digital Identity Processor – Windows Forms

Program.cs

```

namespace Question4;

public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();

        string imgPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
"images", "coat_of_arms.png");
        if (File.Exists(imgPath))
            picCoatOfArms.Image = Image.FromFile(imgPath);
    }

    private void btnValidate_Click(object sender, EventArgs e)
    {
        string name = txtName.Text.Trim();
        string id = txtID.Text.Trim();
        string citizenship = cmbCitizen.SelectedItem?.ToString() ?? "";

        if (string.IsNullOrEmpty(name) ||
string.IsNullOrEmpty(id))
        {
            lblResult.Text = "Please fill in your name and ID number.";
            return;
        }

        var profile = new CitizenProfile(name, id, citizenship);
        lblResult.Text = profile.ValidateID();
    }

    private void btnGenerate_Click(object sender, EventArgs e)
    {
        string name = txtName.Text.Trim();
        string id = txtID.Text.Trim();
        string citizenship = cmbCitizen.SelectedItem?.ToString() ?? "";

        if (string.IsNullOrEmpty(name) ||
string.IsNullOrEmpty(id))
        {
            txtSummary.Text = "Please fill in all fields before generating a
profile.";
            return;
        }

        var profile = new CitizenProfile(name, id, citizenship);
        string validation = profile.ValidateID();

        txtSummary.Text =
            $"==== DIGITAL CITIZEN SUMMARY ==== {Environment.NewLine}" +
            $"Name: {profile.FullName} {Environment.NewLine}" +
            $"ID Number: {profile.IDNumber} {Environment.NewLine}" +
            $"Age: {profile.Age} {Environment.NewLine}" +
            $"Citizenship: {profile.CitizenshipStatus} {Environment.NewLine}"
+
            $"Validation: {validation} {Environment.NewLine}" +
            $"Processed at: Home Affairs Digital Desk {Environment.NewLine}"
+
            $"Timestamp: {DateTime.Now:yyyy/MM/dd HH:mm:ss}";
    }
}

```

Output Screenshots:

Form1

Home Affairs Digital Identity Processor

Enter your Name:

Enter your ID:

Choose your Citizen:

Validate ID

Valid ID. Citizen is 21 years old.

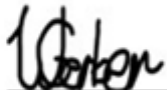
==== DIGITAL CITIZEN SUMMARY ====

Name: Willem
ID Number: 0407055239087
Age: 21
Citizenship: South African
Validation: Valid ID. Citizen is 21 years old.
Processed at: Home Affairs Digital Desk
Timestamp: 2026/04/28 17:15:59

Generate profile

External Evidence:

<https://github.com/Gerber-W/Software-Design-Testing-with-C-SDT621>

Faculty of Information Technology									
I declare that I am familiar with, and will abide to the Examination rules of CTU  Signature	SUBJECT NAME: Solution Development SUBJECT CODE: STD621								
	Formative Assessment Duration: Date: Total Marks: Total pages:					Examiner: Moderator:			
	Student number:								
	2	0	2	3	0	2	5	7	
	Surname: Gerber					Initials: W		/	%

2026/04/28